

EquiHire AI — Semester Project Guide

Problem statement, proposed solution, end-to-end workflow, UI/UX, architecture, and implementation plan

Project basis: This guide is based on the supplied `PROJECT_DOCUMENTATION.md` (EquiHire AI, version 2.4.0). It explains the documented design and adds clearly labeled implementation recommendations where a semester project needs more precise behavior. A feature listed in the source document should not be described as implemented until it has been built and tested.

1. Project overview

Project name: EquiHire AI — Demographic-Blind AI Hiring & Bias Detection Platform

Purpose: Support a more consistent and reviewable technical hiring process by separating identity-related information from the candidate information used for assessment, evaluating job-related skills, and recording assessment and audit results.

The source document describes these main capabilities:

1. Resume upload and anonymization.
2. Technical-domain selection.
3. Timed multiple-choice and aptitude assessment.
4. Coding challenge with test-case verification.
5. Resume skill analysis and gap analysis.
6. Bias/neutrality audit reporting.
7. Candidate results and recruiter review.
8. Role-based accounts and a MongoDB-backed Express API.

Primary users: Candidates, recruiters, and administrators/compliance reviewers. The directory description mentions Candidate, Recruiter, and Admin roles; the workflow also refers to a Compliance Auditor. Decide whether the auditor is a separate role or an Admin permission before implementation.

2. Problem statement

2.1 The problem

Traditional recruitment can be affected by irrelevant identity signals and inconsistent evaluation. Names, photographs, gendered language, age indicators, location, and university reputation may influence how a resume is perceived even when those details do not demonstrate a candidate's ability to perform the job. Manual resume screening can also make it difficult to apply the same assessment criteria to every applicant or to explain why a candidate received a particular result.

For a technical hiring process, another challenge is that a resume alone may not establish practical ability. Recruiters need structured evidence of relevant knowledge, problem-solving, and coding skills.

2.2 Problem statement (ready for a project report)

How might we provide a more consistent and explainable technical candidate assessment process that reduces exposure to demographic and other irrelevant identity cues, evaluates candidates against job-related skills, and gives recruiters auditable evidence for human review?

2.3 Project objectives

- Reduce unnecessary exposure of demographic and identity information during initial assessment.
- Assess candidates against a selected technical domain and a defined job-related rubric.
- Combine resume-derived skill evidence with objective assessment and coding results.
- Record the assessment inputs, outputs, and audit events needed to review how results were produced.
- Provide candidates with clear progress, instructions, and result information.
- Provide recruiters with a searchable, anonymized candidate pool and transparent evidence rather than an unexplained score alone.

2.4 Scope and limits

This is an **assessment-support system**, not an autonomous hiring authority. A score or label should not by itself decide whether someone is hired. Recruiters should review the underlying evidence, accommodations, and job requirements.

Anonymization can reduce exposure to some identity cues, but it cannot guarantee that all bias is removed. Resume content may indirectly reveal identity, and assessment design, scoring, accessibility, and human decisions can also introduce unfairness.

The source document describes EEOC/GDPR audit and compliance goals. A software report or “verified” flag alone does **not** establish legal compliance. Compliance depends on the actual system, jurisdiction, data handling, validation, policies, and review by qualified people.

3. Proposed solution

EquiHire AI proposes a workflow with four connected parts:

A. Privacy-aware candidate intake

The candidate creates an account, uploads a resume, and receives a candidate reference ID. The system extracts relevant career and skill information and creates a redacted assessment view. Identity data, if retained for legitimate account or contact purposes, is kept separate from the evaluator-facing profile.

B. Structured, job-related assessment

The candidate chooses a technical domain. The platform presents a timed MCQ/aptitude section and a coding challenge. Questions, time limits, test cases, and scoring rules are configured in advance and applied consistently.

C. Explainable results and audit record

The system presents component-level results—resume skill evidence, MCQ/aptitude performance, coding evidence—and explains how any combined score was calculated. It records relevant events and the version of the rubric/question set used.

D. Human recruiter review

Recruiters view anonymized profiles, compare job-related evidence against role requirements, and record a human review decision with a reason. Identity details should only be revealed at an appropriate stage and to authorized users.

Important implementation distinction: The source file describes anonymization, AST-based coding evaluation, audit metrics, and composite scores, but does not fully specify their algorithms or validation. Those details must be designed, implemented, and tested; do not treat example defaults or stated metrics as measured results.

4. Roles and permissions

Role	Main responsibilities	Access boundaries
Candidate	Register, upload/update resume, choose domain, take assessments, view own progress/results	Can access only their own records; cannot view other candidates or internal recruiter notes
Recruiter	Define/view job requirements, review anonymized candidate evidence, filter candidates, record review notes	No access to hidden identity fields during blind review; access only to assigned job/candidate pool
Admin	Manage accounts, domains, question bank, configuration, and system health	Administrative access should be logged and limited

Role	Main responsibilities	Access boundaries
Compliance reviewer (optional role)	Inspect audit events, question/rubric versions, and reports	Read-only audit access where possible; no unnecessary candidate identity access

Use server-side authorization for every protected API. Hiding a button in the UI is not access control.

5. Recommended end-to-end workflow

5.1 High-level flow

flowchart TD
 A[Candidate visits platform] --> B[Sign up / sign in]
 B --> C[Candidate dashboard]
 C --> D[Upload resume]
 D --> E[Validate file and obtain consent]
 E --> F[Extract resume text]
 F --> G[Redact identity cues and review redaction]
 G --> H[Create anonymized candidate profile]
 H --> I[Choose job / technical domain]
 I --> J[Read instructions and accommodations]
 J --> K[Timed MCQ and aptitude assessment]
 K --> L[Coding challenge and test cases]
 L --> M[Validate and score submissions]
 M --> N[Generate evidence-based results]
 N --> O[Save assessment and audit events]
 O --> P[Candidate sees status and permitted results]
 O --> Q[Recruiter reviews anonymized evidence]
 Q --> R[Human review and documented next step]
 R --> S[Reveal identity only when authorized and appropriate]

5.2 Detailed candidate journey

Step 1 — Landing page

Page: `index.html`

- Explain the purpose, candidate journey, privacy approach, and limits in plain language.
- Provide clear actions: “Create candidate account,” “Sign in,” and “Recruiter sign in.”
- Show only verified platform statistics. Avoid presenting illustrative counters as live or audited metrics.
- Keep animation decorative and provide a reduced-motion option.

Success state: User understands what the platform does and how to begin.

Step 2 — Account registration and authentication

Pages: `signup.html`, `login.html`

- Candidate enters required account details.
- Recruiter/admin accounts should be provisioned or verified through a controlled process; do not let anyone self-select privileged roles.
- Validate input, hash passwords using a suitable password-hashing library, and create a secure session.
- Redirect each role to its permitted dashboard.
- Provide clear error messages without revealing whether sensitive account details exist.

Success state: Authenticated user reaches the correct dashboard with server-verified permissions.

Step 3 — Candidate dashboard and application setup

Page: `candidate-dashboard.html`

Show a simple progress tracker:

1. Resume
2. Domain
3. MCQ/aptitude
4. Coding
5. Results

Include status, due date if applicable, and a “Continue” action. A candidate should be able to understand what is complete, what remains, and whether a submission is saved.

Step 4 — Resume upload and validation

Page: `upload-resume.html`; **script:** `js/upload.js`; **API:** `POST /api/resume/upload`

- Explain accepted file types and size limits before upload.
- Validate extension, MIME type, file size, and actual file content on the server.
- Show upload progress, success, and actionable errors.
- Obtain clear consent and explain how the resume will be used and retained.
- Avoid putting uploaded resume text in URLs, logs, or client-side error messages.

Success state: A validated resume is stored securely and associated with the candidate account.

Step 5 — Parse and anonymize

- Extract resume text and identify fields that should be hidden from the assessment view.
- Redact direct identifiers such as name, photo, contact details, and address. Review age/date cues, gendered pronouns, and institution prestige carefully; automated redaction may miss indirect clues or remove relevant information.
- Keep the original resume and identity mapping separate from the evaluator-facing profile, with restricted access and retention rules.
- Show the candidate a preview of the redacted profile and allow correction of extraction errors.
- Store which markers were detected/redacted and the processing status.

Success state: Candidate can confirm that the assessment profile is readable and does not unnecessarily expose identity details.

Step 6 — Choose job or technical domain

Page: `select-domain.html`; **data:** `data/domains.json`

- Prefer selecting a specific job opening and its approved rubric. If the project uses domain-only selection, explain that it is a demo approximation.
- Show domain description, required skills, estimated duration, and assessment format.

- Avoid implying that a candidate's choice of domain is itself a measure of merit.

Success state: An assessment is created with a stable domain/job and rubric version.

Step 7 — Instructions and readiness

Before starting the timed section, show:

- Total duration and whether the timer continues after navigation.
- Number and types of questions.
- Whether answers can be changed.
- Autosave/submission behavior.
- Supported accessibility accommodations and a contact path.
- A clear “Start assessment” confirmation.

Do not start the timer before the candidate is ready.

Step 8 — MCQ and aptitude assessment

Page: `assessment.html`; **script:** `js/assessment.js`; **question data:** `data/questions.json`

- Load questions from the server using the assessment ID.
- Display one question or a manageable group at a time, with progress and remaining time.
- Save answers reliably; handle refresh, network interruption, and timeout explicitly.
- Randomize question/option order only if the rubric and answer keys support it.
- Score using a documented answer key and rubric. Do not expose correct answers during an active test unless that is part of the design.

Success state: Answers are submitted once, with a timestamp and a clear confirmation.

Step 9 — Coding challenge

Page: `coding-test.html`; **script:** `js/coding.js`

- Provide problem statement, constraints, examples, language choice, editor, run/test controls, and submission status.
- Run untrusted code in a properly isolated server-side execution environment with CPU, memory, time, and network limits. A browser editor alone is not a secure execution sandbox.
- Show test-case outcomes and useful compiler/runtime errors without exposing hidden tests.
- Use language-appropriate parsing/AST analysis only as one signal; AST shape alone does not prove correctness, efficiency, or fairness.
- Verify algorithmic complexity claims with suitable tests and analysis. Do not promise $O(1)$ performance unless the task and implementation support that claim.

Success state: Final submission is recorded and evaluated against the versioned test suite.

Step 10 — Results calculation

Page: `final-score.html`; **API:** `POST /api/assessment/submit`

Present:

- MCQ/aptitude result.
- Coding result and test evidence.
- Resume skill evidence and gaps.
- Rubric weights and formula, if a composite score is used.
- Limitations, review status, and next step.

Recommended scoring design: Define weights before implementation and show them to candidates where appropriate. Keep component scores visible. Do not use undocumented default values from a database schema as actual candidate scores. Avoid “STRONG FIT” or “RECOMMENDED” as an automatic hiring verdict; use neutral status labels such as “Assessment complete — recruiter review pending.”

Step 11 — Bias/neutrality audit

Page: `bias-report.html`; **script:** `js/bias.js`

The source file proposes linguistic neutrality, demographic leakage, anonymization status, and audit metrics. To make these meaningful:

- Define exactly what each metric measures, its denominator, thresholds, and limitations.
- Record redaction outcomes and question/rubric versions.
- Use a test set with known examples to measure missed redactions and over-redaction.
- Review assessment language for irrelevant demographic cues.
- Treat “0% bias” or “100% compliance” as unsupported unless a clearly defined, validated measurement justifies it. Prefer “checks passed/failed,” evidence, and caveats over absolute claims.

Step 12 — Recruiter review

Page: `recruiter-dashboard.html`; **script:** `js/dashboard.js`

- Show candidates using a pseudonymous reference ID.
- Filter by job, assessment status, and job-related criteria.
- Provide a candidate detail view with skill evidence, component results, rubric, and audit notes.
- Allow a recruiter to record a human review outcome and a reason.
- Keep identity hidden during blind review; log authorized identity reveal.
- Do not sort solely by an opaque composite score. Provide evidence and allow reviewers to inspect relevant components.

Step 13 — Candidate status and closure

- Candidate sees their application/assessment status and permitted feedback.
 - Provide a clear completion message and support/contact route.
 - Apply the documented retention/deletion policy.
 - Log administrative changes and final review actions.
-

6. UI/UX design system

6.1 Visual direction from the source file

The source specifies a light theme using deep emerald green (#116446), pale sage backgrounds, white cards, dark pine text, and muted green secondary text. It also describes a glassmorphic header, a floating circular assistant button, an intro scanner animation, and responsive layouts.

Use these as the starting design direction, but prioritize clarity, accessibility, and trust over decorative effects.

6.2 Suggested design tokens

```
:root {  
  --primary: #116446;  
  --primary-hover: #0C4D36;  
  --primary-light: #1B8A62;  
  --primary-soft: #E8F3EE;  
  --background: #F4F9F6;  
  --surface: #FFFFFF;  
  --border: #D1E3D9;  
  --text: #0C241B;  
  --text-muted: #4A6B5D;  
  
  --radius-card: 16px;  
  --radius-control: 10px;  
  --space-1: 4px;  
  --space-2: 8px;  
  --space-3: 12px;  
  --space-4: 16px;  
  --space-6: 24px;  
  --space-8: 32px;  
}
```

6.3 Layout rules

- Use a consistent centered content area with a readable maximum width.
- Keep candidate tasks single-column where possible; use a two-column layout only when the content benefits from it.
- Use a persistent progress indicator during multi-step candidate flows.
- Keep primary actions visually distinct and consistently placed.
- Use cards for meaningful groups, not every individual text block.
- On mobile, stack columns, keep controls full-width when useful, and prevent horizontal scrolling.

- Make loading, empty, error, success, disabled, and timeout states part of every important screen.

6.4 Typography and components

- Use a clear sans-serif typeface with a small, consistent type scale.
- Use sentence-case labels and concise action text.
- Buttons: primary for the main action, secondary for alternatives, danger only for destructive actions.
- Forms: visible labels, inline validation, and help text near the relevant field.
- Tables: readable column headings, row actions, sorting/filtering controls, and an empty state.
- Status: pair color with text/icon so meaning does not depend on color alone.
- Coding editor: monospace font, visible line numbers if available, clear console/test results, and keyboard-accessible controls.

6.5 Page-by-page UX blueprint

Page	Main content	Primary action	Essential states
Landing	Purpose, steps, privacy summary, FAQs	Create account / Sign in	Loading only for live metrics; no fake counters
Signup/login	Minimal form, role-aware flow	Create account / Sign in	Validation, invalid credentials, server error
Candidate dashboard	Stepper, status cards, next task	Continue	New, in progress, complete, needs attention
Resume upload	File drop zone, requirements, privacy notice	Upload resume	Uploading, invalid file, parsing, redaction preview
Domain selection	Job/domain cards, requirements, duration	Select and continue	No openings, selected, unavailable
MCQ	Question, options, progress, timer	Save/Next/Submit	Answered, unanswered, autosaved, timeout
Coding	Problem, editor, language, tests, console	Run / Submit	Compiling, running, failed tests, submitted
Resume analysis	Skills, evidence, gaps, extraction corrections	Confirm profile	Parsing issue, corrected, confirmed
Bias report	Checks, metric definitions, evidence, caveats	View audit details	Passed, needs review, unavailable
Final results	Component scores, rubric, explanation, status	Return to dashboard	Pending evaluation, complete, review pending
Recruiter dashboard	Filters, anonymized table, metrics	Open candidate	Loading, empty, no matches, access denied
Candidate detail	Skill evidence, results, audit, review notes	Record review	Draft, saved, identity concealed/revealed
Admin	Users, domains, questions, system status	Save configuration	Validation, saved, failed, audit logged

6.6 Accessibility and trust checklist

- Keyboard navigation and visible focus.
- Proper labels, headings, landmarks, and accessible error announcements.

- Sufficient text/background contrast.
 - Screen-reader-friendly timer and progress updates; avoid announcing every second.
 - Reduced-motion support; animations must not block task completion.
 - Clear accommodation process and equivalent assessment handling.
 - Plain-language explanation of data collection, use, retention, and deletion.
 - Never use color alone to communicate score or status.
-

7. System architecture

7.1 Architecture described in the source

flowchart LR
UI[Web client: HTML/CSS/JS and React/Vite frontend] --> API[Node.js / Express REST API]
UI --> AUTH[Auth controller]
API --> RES[Resume controller]
API --> ASM[Assessment controller]
AUTH --> DB[(MongoDB)]
RES --> DB
ASM --> DB
DB --> DATA[domains.json / questions.json / statistics.json]

The source describes both a set of static HTML pages and a `Frontend/` Vite + React application. Clarify whether these are two alternative frontends, a migration, or both served in the final project. Avoid maintaining duplicate implementations without a clear reason.

7.2 Recommended request flow

1. Browser sends an authenticated request to the Express API.
2. Middleware validates session/token and role permissions.
3. Route validates request shape and ownership.
4. Controller invokes the relevant service (resume parsing, assessment, audit).
5. Service reads/writes MongoDB and versioned static configuration.
6. API returns only the fields that the current role is allowed to see.
7. Frontend renders success/error state and updates progress.
8. Security-relevant and assessment events are written to an audit log.

Keep business logic in service/controller modules rather than embedding scoring or access checks only in browser JavaScript.

7.3 API endpoints documented in the source

Method	Endpoint	Purpose
POST	<code>/api/auth/register</code>	Register account
POST	<code>/api/auth/login</code>	Authenticate
POST	<code>/api/resume/upload</code>	Upload and process resume
POST	<code>/api/assessment/submit</code>	Submit assessment results
GET	<code>/api/health</code>	Health check

The source mentions `/api/resume` and `/api/assessment` route groups, but does not specify every endpoint needed by the full workflow. Recommended additions include:

- `GET /api/me` — current authenticated user and role.
- `GET /api/candidate/application` — candidate's own progress.
- `GET /api/domains` or `GET /api/jobs` — available domains/openings.
- `POST /api/assessment/start` — create an assessment attempt.
- `GET /api/assessment/:id` — retrieve permitted assessment state.
- `PUT /api/assessment/:id/answers` — save answers.
- `POST /api/assessment/:id/code-run` — run code in an isolated executor.
- `POST /api/assessment/:id/submit` — finalize attempt.
- `GET /api/candidate/results` — candidate's own results.
- `GET /api/recruiter/candidates` — authorized anonymized pool.
- `POST /api/recruiter/reviews` — save a human review.
- `GET /api/audit/:candidateRef` — authorized audit view.

These are recommendations, not endpoints confirmed to exist in the uploaded file.

8. Database design

The source names the MongoDB database `equihire_ai_db` and these collections: `users`, `candidates`, `assessments`, and `auditlogs`.

8.1 Existing schema concepts in the source

Collection	Fields described
<code>users</code>	<code>name</code> , <code>email</code> , <code>password</code> , <code>role</code> , <code>createdAt</code>
<code>candidates</code>	<code>refId</code> , <code>cloakedName</code> , <code>skills</code> , <code>redactedMarkers</code> , <code>parsedExperienceYears</code> , <code>resumeTextRedacted</code> , <code>uploadedAt</code>
<code>assessments</code>	<code>candidateRef</code> , <code>domainId</code> , <code>mcqScore</code> , <code>codingScore</code> , <code>aptitudeScore</code> , <code>compositeScore</code> , <code>verdict</code> , <code>evaluatedAt</code>
<code>auditlogs</code>	<code>candidateRef</code> , <code>linguisticNeutralityScore</code> , <code>anonymizationStatus</code> , <code>eeocComplianceVerified</code> , <code>genderBiasPercentage</code> , <code>timestamp</code>

The source schema includes example/default score values and example verdicts. These should be removed from production defaults; a new assessment must begin as pending/unscored, not with a fabricated result.

8.2 Recommended data model refinements

- **User:** store a password hash, not a plaintext password. Enforce unique normalized email and role permissions.
- **Candidate:** separate account identity/contact data from the anonymized profile. Use a random internal ID; do not expose sequential or guessable IDs.
- **Resume record:** file storage reference, parse status, redaction version, redaction review status, consent timestamp, retention/deletion status.

- **Assessment:** candidate reference, job/domain ID, rubric version, question-set version, attempt status, start/submit timestamps, component scores, scoring explanation.
- **Answer/submission:** question ID, answer, code submission reference, execution results, timestamps. Restrict access to hidden tests and answer keys.
- **Audit event:** event type, actor role/ID, target record, timestamp, action/result, relevant version identifiers. Avoid placing raw resume text, passwords, or unnecessary personal data in logs.
- **Recruiter review:** reviewer, candidate/assessment reference, job, structured outcome, reason, timestamp, and whether identity was revealed.

Use schema validation, indexes, ownership checks, and retention policies. Define whether deleting an account also deletes or de-identifies its related records.

9. Assessment and scoring workflow

9.1 Separate evidence before combining it

A candidate's result should be built from traceable components:

1. **Resume skill evidence:** extracted skills with supporting resume snippets or experience evidence; show extraction uncertainty.
2. **MCQ/aptitude:** correct answers under a versioned answer key.
3. **Coding:** public/hidden test outcomes, execution limits, and rubric-based code review signals.
4. **Audit checks:** redaction and assessment-language checks. These are process-quality signals, not candidate merit points.

9.2 Composite score

If a composite score is required for the semester demonstration, define a transparent formula before testing. For example:

$$\text{Composite} = (\text{MCQ} \times w1) + (\text{Coding} \times w2) + (\text{Resume evidence} \times w3)$$

$$\text{where } w1 + w2 + w3 = 1.$$

The weights above are placeholders; choose and justify them with the project supervisor and job-related rubric.

Normalize each component to the same scale before combining. Show component scores and the formula so the composite is not a black box.

Do not include demographic data or audit "neutrality" values in the candidate merit score. Do not treat missing data as zero without a documented rule. Use neutral workflow statuses such as:

- Not started
- In progress
- Submitted

- Evaluation pending
- Assessment complete
- Recruiter review pending
- Review completed

10. Bias mitigation and audit design

A meaningful audit should answer: **what was checked, on which data, using which version, what was found, and what remains uncertain?**

Recommended audit categories:

Check	What to record
Redaction coverage	Detected marker types, redacted fields, review outcome, known limitations
Resume readability	Whether relevant skills/experience remain understandable after redaction
Question language	Question-set version, review date, flagged wording, reviewer outcome
Assessment consistency	Rubric/version, time rules, accommodations, scoring implementation
Access control	Identity reveal and privileged access events
Result traceability	Input versions, component results, score formula/version, timestamps
Outcome monitoring	Carefully governed aggregate analysis, sample size, limitations, and review actions

Do not claim “zero bias,” “100% demographic neutrality,” or legal compliance merely because a dashboard displays those numbers. If the project demonstrates metrics, label them as demo/test results and explain the test method and sample.

11. Security, privacy, and reliability requirements

Minimum project checklist:

- Hash passwords with a modern password-hashing algorithm; never store plaintext passwords.
- Use secure session/token handling, expiration, and logout.
- Enforce authorization on the server for candidate, recruiter, admin, and audit routes.
- Validate and safely store uploaded files; restrict file types and sizes.
- Keep original resumes and identity mappings separate from anonymized evaluation data.
- Use an isolated execution environment for submitted code, with strict resource limits and no unnecessary network access.

- Protect secrets with environment variables; do not commit credentials.
 - Use HTTPS in deployment.
 - Avoid logging secrets, raw resumes, or unnecessary personal information.
 - Add rate limiting and sensible error handling to authentication and upload routes.
 - Define data retention, deletion, and backup procedures.
 - Test accessibility and failure cases, not only the happy path.
-

12. Implementation plan for a semester project

Build the workflow in vertical slices, so each phase produces a demonstrable end-to-end feature.

Phase 1 — Requirements and workflow

- Confirm roles, project scope, and whether the frontend is static HTML or React/Vite.
- Define one sample job/domain and its skills/rubric.
- Draw use cases, data flow, and database relationships.
- Define what “anonymized,” “bias check,” and “score” mean in this project.

Deliverable: Approved requirements, workflow diagram, wireframes, and data model.

Phase 2 — Foundation

- Set up frontend, Express server, MongoDB connection, environment configuration.
- Implement authentication and server-side role checks.
- Build shared navigation, form components, status messages, and responsive design tokens.

Deliverable: Login/signup and role-specific dashboard skeleton.

Phase 3 — Resume intake

- Implement secure upload, parsing, redaction preview, candidate reference, and persistence.
- Add extraction/redaction error states and candidate correction flow.

Deliverable: Candidate can upload and review an anonymized profile.

Phase 4 — Domain and MCQ

- Add domain/job selection and versioned question bank.
- Implement assessment start, timer, autosave, submit, and scoring.

Deliverable: Candidate completes an MCQ attempt and sees a recorded result.

Phase 5 — Coding assessment

- Build editor and test UI.
- Integrate a safely isolated execution service or clearly label a limited local prototype.
- Store submissions and test results.

Deliverable: Candidate submits code and receives reproducible test outcomes.

Phase 6 — Results and audit

- Implement component score calculation and explanation.
- Add audit events, redaction checks, and a report with metric definitions and caveats.

Deliverable: Results and audit evidence are linked to a specific assessment attempt.

Phase 7 — Recruiter review

- Build anonymized candidate table, filters, candidate detail, review form, and access logging.
- Add controlled identity reveal only if required by the approved workflow.

Deliverable: Recruiter can review evidence and record a human decision.

Phase 8 — Testing and presentation

- Test normal, invalid, interrupted, unauthorized, and edge-case flows.
- Conduct usability and accessibility checks.
- Prepare sample data, demo script, screenshots, and project report.

Deliverable: Tested semester demo with known limitations documented.

13. Testing plan

Test area	Example test
Authentication	Invalid password; duplicate email; candidate cannot access recruiter API
Upload	Unsupported file, oversized file, corrupted PDF/DOCX, parse failure
Anonymization	Name/contact redacted; indirect identity clues reviewed; relevant skills retained
Assessment	Timer expiration, refresh, answer autosave, duplicate submission
Coding	Correct/incorrect code, timeout, memory limit, malicious input, hidden tests not exposed
Scoring	Formula and rounding verified; missing component handled explicitly
Audit	Event recorded; metric definition visible; no unsupported absolute claims
Recruiter review	Filters work; candidate identity remains concealed; review action is logged
Accessibility	Keyboard-only flow, screen reader labels, contrast, reduced motion

14. Suggested demonstration script

1. Open the landing page and explain the problem and intended solution.
2. Register/sign in as a candidate.
3. Upload a sample resume and show validation and redaction preview.
4. Select a sample technical domain/job.
5. Complete a short MCQ assessment.
6. Submit a coding solution and show test-case results.
7. Open the candidate results and explain each component and the formula.
8. Open the audit report and explain exactly what was checked and what it cannot prove.
9. Sign in as a recruiter and show the anonymized candidate pool.
10. Open a candidate, inspect evidence, and record a human review.
11. Show role-based access by attempting an unauthorized action and confirming it is blocked.

Use synthetic candidate data for demonstrations.

15. Important clarifications before implementation

The supplied document does not fully define the following. Confirm them with your supervisor or project team:

1. Is the final frontend the static HTML/CSS/JS site, the Vite/React app, or a migration to React?
 2. Is “Compliance Auditor” a distinct role or an Admin permission?
 3. Which exact ten domains are in `domains.json`, and what job-related rubric belongs to each?
 4. What resume parser and redaction method will be used? How will missed or incorrect redactions be reviewed?
 5. Where will code execute, and what isolation/security controls are available?
 6. What are the exact scoring weights, normalization rules, and missing-data behavior?
 7. What do the audit metrics measure, and how are they validated?
 8. What data is retained, for how long, and who can access original resumes?
 9. Is the platform a prototype or intended for real candidate decisions? If real use is contemplated, additional legal, security, accessibility, and independent validation work is required.
-

16. Conclusion

EquiHire AI is best presented as a **privacy-aware, structured technical assessment and recruiter decision-support platform**. Its core workflow is:

Register → Upload resume → Parse/redact → Confirm profile → Select job/domain → MCQ/aptitude → Coding → Explainable results → Audit record → Anonymized recruiter review → Human decision.

For a successful semester project, prioritize making this workflow complete, understandable, secure, and testable. A polished interface is valuable, but the central deliverable is a reliable path from candidate input to traceable evidence and a human-reviewed outcome.

Source note

Project-specific names, pages, scripts, endpoints, database collections, colors, and stated features in this guide come from the uploaded `PROJECT_DOCUMENTATION.md`. Sections explicitly labeled “recommended,” “suggested,” or “important clarification” are implementation guidance added to make the project workflow more precise; they are not claims that those features already exist.

